

Platform Data Engineer Python Coding Test Guide

이 문서는 Senior/Lead급 Platform Data Engineer의 Python 코딩 테스트를 준비하기 위한 실전 가이드다.

목표는 Python 문법이나 알고리즘 패턴을 단순히 암기하는 것이 아니다. 제한된 시간 안에 다음 역량을 일관되게 보여주는 것이 목표다.

- 모호한 요구사항을 명확한 contract로 바꾸는 능력
- 올바른 자료구조와 알고리즘을 선택하는 능력
- 경계 조건과 실패 시나리오를 먼저 발견하는 능력
- 메모리 사용이 제한된 streaming 코드를 작성하는 능력
- 테스트 가능하고 확장 가능한 API와 객체를 설계하는 능력
- 동시성, 재시도, 중복 처리, backpressure 같은 production 문제를 설명하는 능력
- 구현의 시간·공간복잡도와 trade-off를 명확하게 전달하는 능력

1. 인터뷰에서 실제로 평가하는 것

Platform Data Engineer 코딩 테스트는 크게 네 층으로 나눌 수 있다.

1.1 Correctness

- 요구사항을 정확히 이해했는가?
- 정상 입력뿐 아니라 empty, duplicate, malformed input을 처리하는가?
- 결과의 순서, 중복, 오류 정책이 contract와 일치하는가?
- 테스트를 통해 구현을 검증하는가?

1.2 Problem solving

- 문제를 작은 단계로 나누는가?
- 자료구조와 알고리즘 선택에 근거가 있는가?
- brute-force에서 더 나은 접근으로 발전시킬 수 있는가?
- 시간·공간복잡도를 정확하게 설명하는가?

1.3 Code design

- 함수와 클래스의 책임이 명확한가?
- 이름과 타입이 의도를 설명하는가?
- 외부 dependency를 분리하여 테스트할 수 있는가?
- 불필요한 abstraction이나 design pattern을 추가하지 않는가?

1.4 Production awareness

- 입력 전체를 메모리에 올리지 않아도 되는가?
- 데이터가 중복되거나 순서가 뒤섞이면 어떻게 되는가?
- 부분 실패, retry, timeout, idempotency를 고려하는가?
- concurrency를 선택할 때 workload와 병목을 구분하는가?
- 로그, metric, checkpoint 등 운영 관점을 설명할 수 있는가?

모든 문제에서 네 층을 같은 깊이로 다룰 필요는 없다. 먼저 정확한 단순 구현을 완성하고, 남은 시간에 production consideration을 설명한다.

2. 권장 학습 우선순위

우선순위	영역	도달 목표
1	Contract, cases, 자료구조, 복잡도	모든 문제를 같은 절차로 시작할 수 있다
2	테스트와 edge case	정상·경계·오류 동작을 코드로 검증할 수 있다
3	Iterable, iterator, generator, batching	큰 입력을 bounded memory로 처리할 수 있다
4	OOP, API design, typing	책임이 분리되고 테스트 가능한 코드를 설계한다
5	예외 처리와 reliability	실패를 분류하고 안전하게 전파·복구한다
6	Thread, process, asyncio, queue	workload에 맞는 concurrency model을 선택한다
7	Memory model, GIL, GC, profiling	runtime 동작을 정확한 조건과 함께 설명한다
8	<code>__slots__</code> 등 세부 최적화	측정 결과를 기반으로 제한적으로 적용한다

GIL, GC, `__slots__` 같은 내부 지식은 중요하지만, 기본 구현과 테스트를 대신하지 않는다. 코딩 테스트에서는 완성도 높은 O(N) 구현과 정확한 edge-case 처리가 세부 runtime 지식보다 먼저다.

3. 모든 코딩 문제에 적용하는 고정 절차

문제를 받으면 바로 코드를 작성하지 않는다. 에디터에 짧은 mini-spec을 작성한다.

```
# Contract
# Input:
# Valid when:
# Return:
# Error when:
# Mutates input?:
#
# Cases
# Normal:
# Boundary:
# Malformed:
#
# Approach:
# Complexity:
```

목표는 긴 문서를 쓰는 것이 아니라, 구현 중 놓치기 쉬운 결정을 밖으로 꺼내는 것이다. 단순한 문제라면 30초, 모호한 문제라면 1~2분 정도로 제한한다.

3.1 Contract

다음 네 가지를 결정한다.

1. Input shape: 어떤 형태와 타입을 받는가?
2. Validity semantics: 무엇이 정상이고 무엇이 invalid인가?
3. Output semantics: 무엇을 어떤 순서와 형식으로 반환하는가?
4. Error policy: 잘못된 사용과 잘못된 데이터를 어떻게 구분하는가?

추가로 입력을 변경할 수 있는지도 확인한다.

예시:

```
# Input: list of candidate integer lists
# Valid when: non-empty, positive integers, no duplicates
# Return: every invalid zero-based index in input order
# Error when: outer input is not a list -> TypeError
# Mutates input?: no
```

질문은 구현에 실제 영향을 주는 것만 한다.

- 첫 오류에서 멈추는가, 모든 오류를 반환하는가?
- 중복은 허용되는가?
- 순서를 보존해야 하는가?
- malformed record 하나가 전체 batch를 실패시켜야 하는가?
- 입력 크기는 어느 정도인가?

인터뷰어가 결정을 맡기면 합리적인 가정을 선언하고 진행한다.

3.2 Cases

무작위로 edge case를 떠올리지 말고 세 범주로 나눈다.

```
Normal      : 일반적인 유효 입력
Boundary    : empty, one item, min/max, duplicate
Malformed   : None, 잘못된 타입, 누락된 필드, 손상된 레코드
```

가능하면 구현 전에 expected result까지 적는다.

```
# [[1, 2], [3]]    -> []
# [[], [1, 1]]     -> [0, 1]
# [[1, "2"], None] -> [0, 1]
# []               -> []
# None             -> TypeError
```

3.3 Design

다음 세 문장으로 설명할 수 있어야 한다.

1. 어떤 순서로 처리할 것인가?
2. 어떤 자료구조를 왜 사용하는가?
3. 시간·공간복잡도는 무엇인가?

예시:

각 내부 리스트를 한 번씩 순회하고, 중복 탐지를 위해 set을 사용하겠습니다. 결과에는 invalid index만 필요하므로 첫 위반을 발견하면 해당 리스트 검사를 중단하겠습니다. 전체 원소 수를 N이라고 하면 평균 $O(N)$ 시간이고, 추가 공간은 가장 큰 내부 리스트의 unique 값 수에 비례합니다.

3.4 Implement

세부 조건보다 외곽 흐름을 먼저 작성한다.

```
def find_invalid_lists(values):
    validate_outer_input(values)

    invalid_indices = []
    for index, candidate in enumerate(values):
        if not is_valid(candidate):
            invalid_indices.append(index)

    return invalid_indices
```

그다음 helper의 세부 규칙을 채운다. 작은 문제에서는 무조건 helper를 만들지 않는다. 함수 분리가 읽기와 테스트를 실제로 개선할 때만 사용한다.

3.5 Verify

앞에서 작성한 example을 그대로 테스트한다.

```
assert find_invalid_lists([[1, 2], [3]]) == []
assert find_invalid_lists([], [1, 1]) == [0, 1]
assert find_invalid_lists([]) == []
```

마지막으로 하나의 입력을 손으로 추적하고 복잡도와 제한사항을 다시 말한다.

3.6 Production follow-up

기본 구현을 완성한 뒤에만 다음을 검토한다.

- 입력이 메모리보다 크다면?
- 중간 실패 후 재시작해야 한다면?
- 결과 순서가 중요하지 않다면 병렬화할 수 있는가?
- 동일 요청이 재시도되면 중복 결과가 생기는가?
- 외부 시스템이 느리거나 응답하지 않으면?
- 어떤 metric과 log가 필요한가?

4. Python 자료구조와 표준 라이브러리

4.1 반드시 알아야 할 복잡도

자료구조/연산	평균 복잡도	주의점
<code>list.append()</code>	$O(1)$ amortized	resize가 발생할 수 있다
<code>list.pop()</code>	$O(1)$	마지막 원소 기준
<code>list.pop(0)</code>	$O(N)$	나머지 원소를 이동해야 한다
<code>deque.append/popleft()</code>	$O(1)$	queue에 적합하다
<code>dict</code> lookup/insert	$O(1)$ average	key는 hashable해야 한다
<code>set</code> membership	$O(1)$ average	dedup과 membership에 적합하다
sorting	$O(N \log N)$	Python sort는 stable하다
<code>heapq</code> push/pop	$O(\log N)$	top-K, scheduler에 적합하다
<code>bisect</code> search	$O(\log N)$	list insertion은 $O(N)$ 이다

4.2 자주 쓰는 도구

```
from collections import Counter, defaultdict, deque
from functools import lru_cache
from itertools import chain, islice
import bisect
import heapq
```

다음 문제 유형을 직접 구현할 수 있어야 한다.

- frequency aggregation: `Counter`, `defaultdict`
- queue/BFS: `deque`
- top-K: `heapq`
- sorted lookup: `bisect`
- deduplication: `set`
- grouping: dictionary of lists/counts
- merge sorted streams: heap
- dependency resolution: graph + topological sort
- interval merge: sort + scan
- bounded recent history: `deque(maxlen=N)`

내부 구현 세부사항보다 어떤 연산이 병목이고 어떤 자료구조가 이를 개선하는지를 설명하는 것이 중요하다.

5. 테스트 작성

테스트는 구현 이후의 부가 작업이 아니라 contract를 실행 가능한 형태로 표현하는 방법이다.

5.1 기본 테스트 범주

범주	확인 내용
Happy path	일반적인 정상 입력
Boundary	empty, one item, 최소·최대값
Invalid input	잘못된 타입과 구조
Duplicate/order	중복 처리와 결과 순서
Exception	정확한 예외 타입과 메시지
State	객체 상태가 예상대로 변경되는가?
Interaction	외부 dependency가 올바르게 호출되는가?

5.2 `assert`의 의미

```
assert actual == expected
```

의미는 “두 값을 비교한다”가 아니라 “이 조건은 반드시 참이어야 한다”이다.

예외 테스트를 직접 작성하면 다음과 같다.

```
try:
    find_invalid_lists(None)
except TypeError:
    pass
else:
    raise AssertionError("Expected TypeError")
```

`pytest`를 사용할 수 있다면 더 명확하게 작성한다.

```
import pytest

def test_non_list_input_raises_type_error():
    with pytest.raises(TypeError, match="values must be a list"):
        find_invalid_lists(None)
```

5.3 Parameterized test

동일한 규칙을 여러 입력에 적용할 때 사용한다.

```
import pytest

@pytest.mark.parametrize(
    ("values", "expected"),
    [
        ([1, 2], [3]),
        ([], [1, 2]),
        ([1, 1], [2, 3]),
        ([0], [-1]),
    ],
)
def test_find_invalid_lists(values, expected):
    assert find_invalid_lists(values) == expected
```

5.4 좋은 테스트의 특징

- 하나의 동작을 명확히 검증한다.
- 이름만 봐도 실패 조건을 알 수 있다.
- 구현 세부사항보다 외부 contract를 테스트한다.
- 실행 순서와 외부 환경에 의존하지 않는다.
- network와 DB 같은 경계는 fake/mock으로 분리한다.
- 정상 사례보다 실패와 경계 사례를 의도적으로 포함한다.

5.5 테스트에서 피할 것

- `except Exception: pass`로 모든 오류를 숨기기
- 함수가 정상 반환했는지 확인하지 않는 예외 테스트
- private method 구현에 지나치게 결합된 테스트
- 시간에 민감하고 비결정적인 sleep 기반 테스트
- 실제 production DB나 API를 unit test에서 호출하기

6. Iterable, Iterator, Generator와 Streaming

6.1 개념 관계

```
Iterable
└─ __iter__()를 통해 iterator를 제공

Iterator
├─ __iter__()
└─ __next__()

Generator
└─ iterator를 간단하게 만드는 함수/객체
```

타입 signature도 의도를 표현한다.

```
from collections.abc import Iterable, Iterator

def parse_lines(lines: Iterable[str]) -> Iterator[dict[str, object]]:
    for line in lines:
        yield parse_line(line)
```

6.2 Generator가 보장하지 않는 것

Generator는 결과 전체를 자동으로 materialize하지 않는다. 그러나 다음이 있으면 전체 공간은 $O(1)$ 이 아닐 수 있다.

- downstream에서 `list(generator)` 호출
- global dedup을 위한 증가하는 `set`
- unbounded cache
- queue producer가 consumer보다 빠름
- generator가 큰 객체를 참조한 상태로 오래 유지됨

따라서 “generator이므로 $O(1)$ ”이 아니라 어떤 상태가 입력 크기에 따라 증가하는지를 분석한다.

6.3 Batching

외부 시스템에 한 건씩 요청하면 overhead가 크고, 전체를 한 번에 보내면 메모리와 failure blast radius가 커진다. 제한된 batch는 두 극단의 균형점이다.

```
from collections.abc import Iterable, Iterator
from itertools import islice
from typing import TypeVar

T = TypeVar("T")

def batched(items: Iterable[T], size: int) -> Iterator[list[T]]:
    if size <= 0:
        raise ValueError("size must be positive")

    iterator = iter(items)

    while batch := list(islice(iterator, size)):
        yield batch
```

테스트:

```
def test_batched_splits_input():
    assert list(batched(range(5), 2)) == [[0, 1], [2, 3], [4]]
```

```
def test_batched_accepts_generator():
    source = (value for value in range(3))
    assert list(batched(source, 2)) == [[0, 1], [2]]

def test_batched_rejects_invalid_size():
    with pytest.raises(ValueError):
        list(batched([1, 2], 0))
```

6.4 Platform 관점의 streaming 질문

- record 순서를 보존해야 하는가?
- malformed record를 skip, quarantine, fail 중 어떻게 처리하는가?
- dedup 범위가 batch 내부인가, 전체 실행인가, 영구적인가?
- producer가 consumer보다 빠르면 어디에서 제한하는가?
- checkpoint는 record, offset, file, partition 중 어느 grain인가?
- retry 후 이미 저장된 record는 어떻게 처리하는가?

7. OOP Recap

OOP는 design pattern을 많이 사용하는 것이 아니라 다음을 명확하게 만드는 방법이다.

State + Behavior + Responsibility + Collaboration

7.1 Encapsulation

객체가 자신의 상태와 invariant를 책임진다.

```
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)
class Record:
    record_id: str
    payload: bytes

    def __post_init__(self) -> None:
        if not self.record_id:
            raise ValueError("record_id must not be empty")
```

`frozen=True`는 attribute 재할당을 제한하지만 deep immutability를 보장하지는 않는다. mutable field 내부는 변경될 수 있다.

7.2 Composition over inheritance

깊은 상속 계층보다 작은 component를 조립하는 방식을 우선 검토한다.

```
Pipeline has a Parser
Pipeline has a Sink
```

상속은 진정한 **is-a** 관계이고 하위 타입이 상위 타입의 contract를 지킬 때 사용한다.

7.3 Protocol과 polymorphism

구체적인 구현보다 필요한 동작에 의존한다.

```
from typing import Protocol

class Parser(Protocol):
    def parse(self, raw: str) -> Record:
        ...

class Sink(Protocol):
    def write(self, record: Record) -> None:
        ...
```

Protocol은 명시적인 상속 없이 method signature를 만족하는 구현을 사용할 수 있는 structural typing이다.

ABC는 공통 구현이나 명시적인 nominal hierarchy가 실제로 필요할 때 고려한다.

7.4 Dependency injection

외부 dependency를 클래스 내부에서 직접 생성하지 않고 constructor나 method parameter로 전달한다.

```
class Pipeline:
    def __init__(self, parser: Parser, sink: Sink) -> None:
        self._parser = parser
        self._sink = sink

    def run(self, lines: Iterable[str]) -> int:
        processed = 0

        for line in lines:
            record = self._parser.parse(line)
            self._sink.write(record)
            processed += 1

        return processed
```

장점:

- parser와 sink를 독립적으로 교체할 수 있다.
- 실제 DB 없이 unit test를 작성할 수 있다.
- orchestration과 외부 I/O 책임이 분리된다.

7.5 Test double을 이용한 OOP 테스트

```
class StubParser:
    def parse(self, raw: str) -> Record:
        return Record(record_id=raw, payload=b"payload")

class InMemorySink:
    def __init__(self) -> None:
        self.records: list[Record] = []

    def write(self, record: Record) -> None:
        self.records.append(record)
```

```
def test_pipeline_writes_every_parsed_record():
    sink = InMemorySink()
    pipeline = Pipeline(parser=StubParser(), sink=sink)

    processed = pipeline.run(["a", "b"])

    assert processed == 2
    assert [record.record_id for record in sink.records] == ["a", "b"]
```

7.6 Class와 instance attribute

```
class Worker:
    worker_type = "batch" # class attribute: shared

    def __init__(self, name: str) -> None:
        self.name = name # instance attribute: per object
        self.tasks = [] # per-object mutable state
```

다음과 같은 mutable class attribute는 대부분 버그다.

```
class BadWorker:
    tasks = [] # 모든 instance가 같은 list를 공유한다
```

7.7 Method 종류

```
class Job:
    def run(self) -> None:
        # instance state를 사용한다
        ...

    @classmethod
    def from_config(cls, config: dict[str, object]) -> "Job":
        # alternative constructor
        return cls()

    @staticmethod
    def is_valid_name(name: str) -> bool:
        # instance/class state가 필요 없는 관련 utility
        return bool(name.strip())
```

클래스와 직접적인 관련이 없는 utility는 `staticmethod`보다 module-level function이 더 단순할 수 있다.

7.8 Equality와 hashability

반드시 지켜야 할 contract:

`a == b`라면 `hash(a) == hash(b)`여야 한다.

해시가 같다고 두 객체가 반드시 같은 것은 아니다. hash key로 사용하는 객체의 equality 의미가 변하면 dictionary/set lookup이 깨질 수 있으므로 immutable value object가 안전하다.

```
@dataclass(frozen=True)
class PartitionKey:
    table: str
    partition_date: str
```

7.9 Inheritance, `super()`, MRO

알아야 할 내용:

- overriding과 base contract
- `super()`가 단순히 "부모 클래스 호출" 이상의 MRO 기반 호출이라는 점
- multiple inheritance의 method resolution order
- 하위 타입이 상위 타입을 안전하게 대체해야 한다는 원칙

인터뷰에서는 코드 재사용만을 위해 깊은 상속 구조를 만들지 않는다. Strategy, Adapter, Factory도 실제 변경 지점이 있을 때만 사용한다.

7.10 OOP 문제 접근 순서

1. 어떤 객체 또는 역할이 존재하는가?
2. 각 역할이 소유하는 상태는 무엇인가?
3. 각 역할의 한 가지 핵심 책임은 무엇인가?
4. 무엇이 변경되거나 교체될 가능성이 있는가?
5. 외부 시스템 경계는 무엇인가?
6. lifecycle과 실패는 누가 책임지는가?
7. 실제 DB/API 없이 어떻게 테스트할 것인가?

예를 들어 pipeline 문제는 다음처럼 나눌 수 있다.

```
Value object      : Record, PartitionKey
Boundary         : Source, Sink
Policy           : Transformer, RetryPolicy
Orchestration    : Pipeline
Failure model    : ParseError, SinkError
Lifecycle        : context manager
```

8. Context Manager와 Resource Safety

파일, connection, lock 같은 resource는 GC에 맡기지 않고 결정론적으로 정리한다.

```
from collections.abc import Iterator
from contextlib import contextmanager

@contextmanager
def managed_resource() -> Iterator[None]:
    # setup
    try:
        yield
    finally:
        # teardown: 성공/실패와 무관하게 실행
        pass
```

알아야 할 내용:

- `__enter__`, `__exit__`
- `try/finally`
- exception이 발생해도 cleanup이 실행되는 이유
- `__exit__` 반환값에 따른 exception suppression
- 여러 resource의 중첩과 `contextlib.ExitStack`
- sync context manager와 async context manager의 차이

9. Decorator

Decorator는 logging, metric, authorization 같은 cross-cutting concern에 사용할 수 있다.

```
from collections.abc import Callable
from functools import wraps
from typing import ParamSpec, TypeVar

P = ParamSpec("P")
R = TypeVar("R")

def log_execution(func: Callable[P, R]) -> Callable[P, R]:
    @wraps(func)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        return func(*args, **kwargs)

    return wrapper
```

확인할 사항:

- `functools.wraps`
- argument가 있는 decorator
- closure와 late binding
- sync 함수와 async 함수의 차이
- retry decorator가 retry할 exception을 제한하는지
- decorator가 함수의 책임과 제어 흐름을 지나치게 숨기지 않는지

10. 예외 처리와 Reliability

10.1 예외를 분류한다

```
Caller/programming error : 잘못된 타입, 필수 argument 누락
Invalid data              : schema 위반, parsing 불가
Transient failure         : timeout, 일시적 network 오류
Permanent external error : 인증 실패, 잘못된 endpoint
Internal invariant error  : 발생하면 안 되는 상태
```

모든 예외를 동일하게 retry하거나 swallow하지 않는다.

10.2 좁게 catch한다

좋지 않은 코드:

```
try:
    process(record)
```

```
except Exception:
    pass
```

호출자가 처리할 수 있는 의미 있는 예외로 변환할 때는 chaining을 유지한다.

```
class PipelineError(Exception):
    """Base exception for pipeline failures."""

class RecordParseError(PipelineError):
    pass
```

```
try:
    record = parse(raw)
except (KeyError, ValueError, TypeError) as error:
    raise RecordParseError("Invalid record") from error
```

10.3 Retry checklist

- 이 오류가 retryable한가?
- 최대 시도 횟수는 있는가?
- exponential backoff와 jitter가 있는가?
- 각 시도에 timeout이 있는가?
- 전체 operation deadline이 있는가?
- retry가 중복 write를 만들지 않는가?
- cancellation과 shutdown에 반응하는가?

10.4 Idempotency

동일 작업을 두 번 실행해도 최종 결과가 한 번 실행한 것과 같아야 하는 상황이 많다.

가능한 방법:

- deterministic idempotency key
- database unique constraint
- compare-and-set/version check
- transactional write
- checkpoint와 committed offset
- overwrite 가능한 partition 단위 처리

“exactly once”라는 표현을 사용할 때는 source read, processing, sink commit 전체 범위를 명확히 한다.

11. Concurrency와 GIL

11.1 선택 기준

상황	우선 검토	주요 비용/위험
Blocking I/O 수십~수백 개	Thread pool	shared state, thread safety
Pure Python CPU-bound	Process pool	serialization, startup, memory
매우 많은 non-blocking I/O	asyncio	cancellation, event-loop blocking
Native library 계산	library 동작 확인	GIL release 여부, native threads

기본 GIL-enabled CPython에서는 한 프로세스 안에서 하나의 thread만 Python bytecode를 실행한다. 하지만 native extension은 GIL을 해제할 수 있고 Python 3.13 이상에는 optional free-threaded build도 존재한다. 따라서 “thread는 CPU-bound에 절대 사용할 수 없다”보다 runtime과 workload를 조건으로 설명한다.

11.2 Thread

알아야 할 내용:

- race condition
- **Lock, RLock, Semaphore, Event**
- thread-safe **queue.Queue**
- shared mutable state 최소화
- future의 exception 확인
- bounded worker pool

11.3 Process

알아야 할 내용:

- process 간 메모리 격리
- argument/result serialization
- process startup 비용
- 너무 작은 task의 parallelization overhead
- worker crash와 partial result
- 플랫폼별 process start method 차이

11.4 asyncio

알아야 할 내용:

- cooperative scheduling
- blocking 함수를 event loop에서 직접 호출하지 않기
- timeout과 cancellation
- semaphore를 통한 concurrency 제한
- **TaskGroup**을 이용한 structured concurrency
- async client/library가 실제로 non-blocking인지 확인
- producer/consumer 간 backpressure

11.5 선택 질문

Concurrency 모델을 고르기 전에 질문한다.

병목은 CPU, network, disk, lock 중 무엇인가?
 작업 단위는 얼마나 큰가?
 결과 순서를 보존해야 하는가?
 공유 상태가 있는가?
 실패 시 다른 작업을 취소해야 하는가?
 동시 요청 수를 어디에서 제한하는가?

12. Memory와 CPython Runtime

12.1 먼저 측정한다

최적화 순서:

1. 입력 크기와 peak memory를 측정한다.
2. 전체 materialization을 제거한다.
3. batching과 data representation을 개선한다.
4. 불필요한 copy와 retained reference를 찾는다.
5. 필요할 때만 object-level 최적화를 적용한다.

`__slots__`를 적용하기 전에 Python 객체 수백만 개를 생성하는 설계 자체가 필요한지 먼저 검토한다.

12.2 Reference counting과 cyclic GC

일반적인 GIL-enabled CPython에서는 reference counting이 주된 객체 생명주기 메커니즘이고, cyclic GC가 순환 참조를 처리한다. 이것은 Python 언어 전체가 아니라 CPython 구현 관점의 설명이다.

다음은 분리해서 이해한다.

Resource cleanup	→ with / try-finally
Object lifetime tracking	→ reference counting / GC
Non-owning reference	→ weakref
Memory optimization	→ measurement, batching, representation

12.3 `__slots__`

장점:

- instance `__dict__` 생성을 피할 수 있다.
- 객체 수가 많을 때 메모리를 줄일 수 있다.
- attribute 접근이 개선될 수 있다.

주의점:

- inheritance 시 subclass도 slots를 정의해야 기대한 효과를 얻는다.
- dynamic attribute 추가가 제한된다.
- weak reference가 필요하면 별도 지원이 필요하다.
- 절감 비율은 객체와 runtime에 따라 다르므로 측정해야 한다.

일반적으로는 직접 `__slots__`를 작성하기보다 다음 형태가 읽기 쉽다.

```
@dataclass(slots=True)
class Record:
    record_id: str
    timestamp: int
```

13. Data Platform 특화 문제 체크리스트

Data semantics

- event time과 processing time 구분
- timezone-aware datetime
- duplicate와 idempotency key
- out-of-order/late event
- schema validation과 schema evolution
- null과 missing field의 차이
- deterministic ordering

Processing

- streaming과 batch 경계
- batch size trade-off
- partitioning key
- checkpoint grain
- partial failure
- retry와 poison record
- bounded memory

External systems

- pagination 종료 조건
- timeout
- rate limit
- exponential backoff와 jitter
- connection cleanup
- transaction boundary
- API/DB 호출 mocking

Operations

- processed/succeeded/failed/skipped count
- latency와 throughput
- retry count
- queue depth
- structured logging

- correlation/job/run ID
- graceful shutdown

14. 실전 연습 방법론

14.1 한 문제의 45분 구성

시간	작업
0~3분	Contract, cases, assumptions 작성
3~7분	접근 방법, 자료구조, 복잡도 설명
7~27분	가장 단순하고 정확한 구현
27~35분	테스트와 bug 수정
35~40분	complexity와 trade-off 설명
40~45분	production follow-up 한 가지 적용

처음부터 production-grade framework를 만들지 않는다. 먼저 working solution을 완성한다.

14.2 연습 세션 규칙

1. 처음 2~3분은 코드를 쓰지 않고 mini-spec만 적는다.
2. 생각 전체가 아니라 결정과 이유를 소리 내어 말한다.
3. happy path를 구현한 뒤 edge case를 하나씩 추가한다.
4. 최소 세 개의 test를 작성한다.
5. 종료 후 코드를 다시 쓰지 말고 문제 해결 과정을 평가한다.

14.3 Self-review rubric

각 항목을 0~2점으로 평가한다.

영역	0점	1점	2점
Contract	바로 코딩	일부 가정 확인	입출력·오류·변경 여부 명확
Cases	happy path만	일부 edge case	정상·경계·malformed 포함
Correctness	주요 버그	수정 후 동작	테스트로 일관되게 검증
Complexity	설명 못함	대략 설명	병목과 trade-off까지 정확
Code design	책임 혼합	부분 분리	단순하고 테스트 가능
Communication	침묵/중계	접근 설명	결정과 이유를 간결히 설명
Production	고려 없음	하나 언급	failure/scale를 구체적으로 연결

총점보다 0점 항목이 반복되는지를 추적한다.

14.4 오답 노트 형식

Problem:
Missed contract:
Missed edge case:
Wrong assumption:
Data structure chosen:
Bug root cause:
Test that would have caught it:
Better explanation:
Review date:

코드 정답 전체를 복사하기보다 놓친 판단과 이를 잡아낼 테스트를 기록한다.

14.5 문제 유형별 연습 목록

Core Python

- nested records validation
- grouping과 aggregation
- duplicate detection
- interval merge
- top-K frequent records
- merge sorted iterators
- dependency ordering

Streaming

- 큰 로그 파일 line-by-line parsing
- fixed-size batch iterator
- rolling aggregation
- bounded-memory deduplication의 trade-off
- multi-file merge
- checkpoint 가능한 iterator

OOP/API design

- pluggable parser와 sink가 있는 pipeline
- retry policy를 주입할 수 있는 API client
- multiple storage backend adapter
- in-memory cache와 TTL policy
- task scheduler와 job state model

Reliability/concurrency

- retry with backoff
- rate limiter
- bounded producer/consumer queue
- concurrent API fetch with result ordering

- graceful worker shutdown
- partial batch failure reporting

14.6 주간 학습 루프

Day 1: 개념 + 작은 구현
Day 2: 동일 개념의 변형 문제
Day 3: 테스트와 edge case 강화
Day 4: 제한 시간 모의 인터뷰
Day 5: 오답 원인 재구현
Day 6: 두 개념을 결합한 platform 문제
Day 7: 휴식 또는 20분 lightweight review

매일 새로운 문제만 풀지 않는다. 48시간 뒤와 1주일 뒤에 힌트 없이 재구현하여 retrieval을 연습한다.

15. Deep Dive 권장 순서

Part 1. Contract와 테스트

- 모호한 요구사항 질문하기
- example을 executable test로 전환하기
- exception과 malformed input 테스트
- parameterized test

Part 2. 자료구조와 Python 표준 라이브러리

- dict/set/deque/heap
- grouping, dedup, top-K
- complexity reasoning

Part 3. Iterator, generator, streaming, batching

- iterator protocol
- lazy pipeline
- bounded memory
- streaming failure handling

Part 4. OOP와 API 설계

- dataclass와 value object
- composition
- Protocol과 dependency injection
- test double
- context manager

Part 5. 예외 처리와 reliability

- exception hierarchy

- chaining
- retry 분류
- timeout, idempotency, checkpoint

Part 6. Concurrency

- thread/process/async 선택
- race condition과 synchronization
- queue와 backpressure
- cancellation과 partial failure

Part 7. Memory와 runtime

- object lifetime
- GC와 circular reference
- GIL과 free-threaded Python
- profiling과 `__slots__`

Part 8. 종합 모의 인터뷰

- 45~60분 제한
- 요구사항 변경 follow-up
- code review
- production extension

16. 시험 직전 5분 체크리스트

문제를 받은 직후

- ☐ 결과 한 단위(grain)는 무엇인가?
- ☐ 입력과 출력 contract는 무엇인가?
- ☐ 중복과 순서 규칙은 무엇인가?
- ☐ empty와 malformed input은 어떻게 처리하는가?
- ☐ 입력을 변경해도 되는가?

구현 전

- ☐ 가장 단순한 정답 접근은 무엇인가?
- ☐ 필요한 자료구조와 선택 이유는 무엇인가?
- ☐ 전체 입력을 메모리에 올리고 있지는 않은가?
- ☐ 시간·공간복잡도는 무엇인가?

구현 후

- ☐ normal case를 실행했는가?
- ☐ empty/boundary case를 실행했는가?
- ☐ malformed/error case를 실행했는가?
- ☐ duplicate와 ordering을 확인했는가?
- ☐ 예외를 너무 넓게 잡거나 숨기지 않았는가?

추가 질문

- ☐ 입력이 100배 커지면 병목은 어디인가?
- ☐ 중간 실패 후 안전하게 재시작할 수 있는가?
- ☐ retry가 duplicate를 만들 수 있는가?
- ☐ concurrency를 제한할 장치가 있는가?
- ☐ 어떤 log와 metric이 필요한가?

17. 핵심 원칙

1. 빠른 코딩보다 빠른 구조화가 먼저다.
2. Contract를 주석과 example로 고정한다.
3. 분모가 되는 기본 데이터 집합 또는 입력 grain을 먼저 만든다.
4. 가장 단순하고 정확한 구현을 먼저 완성한다.
5. 테스트는 contract를 확인하는 실행 가능한 문서다.
6. Generator는 materialization을 피하지만 자동으로 O(1) 공간을 보장하지 않는다.
7. OOP의 목적은 class 수를 늘리는 것이 아니라 책임과 dependency를 분리하는 것이다.
8. Composition과 dependency injection은 확장성과 testability를 함께 개선한다.
9. 모든 오류를 retry하지 않으며, retry 전 idempotency를 확인한다.
10. Concurrency model은 유행이 아니라 실제 병목을 기준으로 선택한다.
11. 최적화는 측정 이후에 한다.
12. Senior답게 보이는 가장 강한 신호는 복잡한 코드가 아니라 명확한 판단과 trade-off다.

참고 자료

- [Python data model](#)
- [Python typing and Protocol](#)
- [Python dataclasses](#)
- [Python collections](#)
- [Python asyncio tasks and TaskGroup](#)
- [Python threading](#)
- [Python free-threading](#)
- [Python exceptions](#)
- [pytest documentation](#)